



ÉCOLE NATIONALE SUPÉRIEURE
D'INFORMATIQUE ET D'ANALYSE DES SYSTÈMES
- RABAT

PROJET DE STAGE

Conception et développement d'une
application web intelligente pour la
gestion et l'analyse automatisées des
factures

Réalisé par :
AMERKAD Soumia

Jury :

Encadrant :
Mr. Anass MEUDOUBI

Année Universitaire 2026-2027

Résumé

Ce rapport présente la conception et le développement de **InvoiceAI**, une application web dédiée à l'automatisation de la gestion et de l'analyse des factures fournisseurs. L'objectif principal est de faciliter l'enregistrement, la classification et l'exploitation des données contenues dans les factures grâce à l'intégration de l'intelligence artificielle.

La solution développée repose sur une architecture client-serveur composée d'un frontend réalisé avec **Next.js** et **TypeScript** et d'un backend développé avec **ASP.NET Core**. Les données sont stockées dans une base **SQL Server** à l'aide de **Entity Framework Core**. L'extraction automatique des informations des factures est assurée par l'**API Google Gemini**, tandis que l'authentification est sécurisée par des **JSON Web Tokens (JWT)**. L'application permet notamment de gérer les catégories, d'extraire les informations des factures, de les vérifier avant leur enregistrement, de consulter les dépenses et de générer des rapports Excel.

Une étude comparative de plusieurs plateformes d'intelligence artificielle a également été réalisée afin de sélectionner la solution la mieux adaptée aux besoins du projet. Des expérimentations sur différents modèles Gemini ont ensuite permis de choisir le modèle utilisé pour le traitement des factures.

Ce projet a ainsi permis de mettre en pratique des compétences en développement full-stack, en intégration d'intelligence artificielle, en gestion des données et en conception d'applications web sécurisées.

Mots-clés : Intelligence artificielle, gestion des factures, extraction automatique, Gemini, ASP.NET Core, Next.js, SQL Server, développement full-stack.

Table des matières

Résumé	1
Liste des Abréviations	7
Introduction générale	8
1 Contexte général	10
1.1 Environnement du stage	11
1.1.1 Présentation de SO SOLUTION	11
1.1.2 Domaines d'activité	12
1.1.3 Fiche technique	13
1.1.4 L'organigramme de SO SOLUTION	13
1.2 Présentation du projet	13
1.2.1 Problématique	13
1.2.2 Solutions proposées	14
1.3 Planification du projet	15
1.3.1 Méthodologie de travail : Agile Scrum	15
1.3.2 Diagramme de Gantt	15
1.4 Équipe de travail	16
1.5 Conclusion	16
2 Analyse et Conception	17
2.1 Introduction	18
2.2 Analyse des besoins	18
2.2.1 Besoins fonctionnels	18

2.2.2	Besoins non fonctionnels	21
2.2.3	Acteurs du système	22
2.2.4	Diagramme de cas d'utilisation	23
2.3	Conception	25
2.3.1	Architecture générale	25
2.3.2	Modélisation du système	26
2.3.3	Étude comparative des plateformes d'intelligence artificielle	29
2.3.4	Comparaison des modèles Gemini	33
2.3.5	Conception des principaux processus	37
2.4	Conclusion	41
3	Réalisation	42
3.1	Introduction	43
3.2	Outils utilisés	43
3.2.1	Visual Studio Code	43
3.2.2	SQL Server Management Studio	43
3.2.3	Postman	44
3.2.4	Google AI Studio	44
3.3	Technologies utilisées	44
3.4	Implémentation de la solution	45
3.4.1	Implémentation de l'authentification	45
3.4.2	Implémentation de la gestion des catégories	46
3.4.3	Implémentation du traitement automatique des factures	46
3.4.4	Implémentation de la gestion des factures	47
3.4.5	Implémentation du tableau de bord et des exports	47
3.5	Présentation de l'application	47
3.5.1	Page d'accueil	48
3.5.2	Inscription et définition des catégories	50
3.5.3	Tableau de bord	51

3.5.4	Ajout d'une facture	52
3.5.5	Liste des factures	53
3.5.6	Analyse par fournisseur	54
3.6	Conclusion	54
Conclusion et Perspectives		56
Webographie		57

Table des figures

1.1	Diagramme de Gantt du projet selon une approche Agile Scrum	16
2.1	Diagramme de cas d'utilisation de l'application	24
2.2	Architecture générale de l'application	26
2.3	Diagramme de classes de l'application	27
2.4	Évolution de la consommation des tokens observée lors des expérimentations.	35
2.5	Diagramme de séquence du traitement d'une facture	38
2.6	Diagramme de séquence de l'authentification	39
2.7	Diagramme de séquence de la gestion des catégories	41
3.1	Logo de Visual Studio Code	43
3.2	Logo de SQL Server Management Studio	43
3.3	Logo de Postman	44
3.4	Logo de Google AI Studio	44
3.5	Page d'accueil d'InvoiceAI	49
3.6	Page d'inscription et définition des catégories	51
3.7	Tableau de bord	52
3.8	Ajout d'une facture avec extraction automatique	53
3.9	Liste des factures	53
3.10	Analyse des dépenses par fournisseur	54

Liste des tableaux

1.1	Fiche technique de SO SOLUTION	13
2.1	Critères d'évaluation des solutions d'intelligence artificielle	29
2.2	Comparaison des plateformes d'intelligence artificielle selon les critères re- tenus	30
2.3	Résultats des expérimentations sur les modèles Gemini	33
2.4	Estimation du coût de traitement d'une facture selon le modèle	36
3.1	Technologies utilisées dans InvoiceAI et leur rôle	45

Liste des Abréviations

API Application Programming Interface

BTP Bâtiment et Travaux Publics

CSV Comma-Separated Values

CSS Cascading Style Sheets

EF Core Entity Framework Core

ERP Enterprise Resource Planning

FEC Fichier des Écritures Comptables

HT Hors Taxes

HTTP HyperText Transfer Protocol

IA Intelligence Artificielle

JPG Joint Photographic Experts Group

JSON JavaScript Object Notation

JWT JSON Web Token

OCR Optical Character Recognition

PDF Portable Document Format

PNG Portable Network Graphics

SQL Structured Query Language

TTC Toutes Taxes Comprises

TVA Taxe sur la Valeur Ajoutée

Introduction générale

La gestion des factures fournisseurs constitue une activité importante pour de nombreuses organisations, mais leur traitement manuel peut être long et source d'erreurs. La saisie des informations, la classification des dépenses et l'exploitation des données contenues dans les factures nécessitent souvent des opérations répétitives. Dans ce contexte, l'intelligence artificielle offre de nouvelles possibilités pour automatiser certaines de ces tâches et faciliter la gestion des informations financières.

C'est dans ce contexte que s'inscrit le projet **InvoiceAI**, réalisé dans le cadre d'un stage de six semaines. L'objectif de ce projet est de concevoir et développer une application web permettant d'automatiser l'extraction et la classification des informations contenues dans les factures fournisseurs, tout en offrant des fonctionnalités de consultation et d'analyse des dépenses.

La solution développée repose sur une architecture client-serveur. Le frontend a été réalisé avec **Next.js** et **TypeScript**, tandis que le backend repose sur **ASP.NET Core**. Les données sont stockées dans une base **SQL Server** et gérées à l'aide de **Entity Framework Core**. L'intégration de l'**API Google Gemini** permet d'extraire automatiquement les principales informations des factures et de proposer leur classification selon les catégories définies par l'utilisateur. L'application intègre également un système d'authentification basé sur les **JSON Web Tokens (JWT)** ainsi que des fonctionnalités d'analyse et d'export des données.

Une étude comparative des solutions d'intelligence artificielle a été réalisée afin d'identifier la plateforme et le modèle les plus adaptés aux besoins du projet. Cette étude a été complétée par des expérimentations portant notamment sur la qualité de l'extraction, la classification, la disponibilité et le coût des modèles Gemini.

Ce rapport présente les différentes étapes de réalisation du projet. Le premier chapitre est consacré à la présentation du contexte du stage et du projet. Le deuxième chapitre porte sur l'analyse des besoins et la conception de la solution. Le troisième chapitre pré-

sente la réalisation de l'application, les technologies utilisées et ses principales interfaces. Enfin, une conclusion générale présente les principaux apports du projet ainsi que les perspectives d'amélioration envisagées.

Chapitre 1

Contexte général

Introduction

Dans ce premier chapitre, nous plongeons dans le contexte de notre projet, axé sur l'automatisation de la gestion des factures fournisseurs grâce à l'intelligence artificielle. Notre initiative consiste à développer une application web permettant l'extraction, la classification et le suivi automatiques des factures. Cette évolution s'inscrit dans un environnement de travail dynamique au sein de **SO Solutions**. Notre objectif est de faciliter la gestion des dépenses, de réduire le temps consacré à la saisie manuelle, et de fournir une vision consolidée des factures fournisseurs. Cette introduction pose les fondations essentielles pour la compréhension de notre projet.

1.1 Environnement du stage

1.1.1 Présentation de SO SOLUTION

SO SOLUTION est un éditeur marocain de solutions digitales de gestion, spécialisé dans le développement de logiciels destinés à accompagner les entreprises dans la digitalisation et l'automatisation de leurs processus de gestion. Forte de **30 ans d'expérience**, l'entreprise met son expertise au service des organisations marocaines en proposant des solutions adaptées aux exigences réglementaires et aux spécificités des différents secteurs d'activité.

L'entreprise se positionne comme un acteur de la transformation numérique des entreprises, en proposant des solutions modulaires, intuitives et évolutives permettant de centraliser et d'automatiser les principaux processus opérationnels. Sa mission consiste notamment à simplifier la gestion des entreprises, à réduire les tâches manuelles répétitives et à fournir aux utilisateurs une meilleure visibilité sur leurs activités.

SO SOLUTION accorde également une importance particulière à la **conformité légale**, à l'**intégration des processus**, à l'**automatisation**, à la **réduction des coûts** et à l'**amélioration de la collaboration**. Ses solutions sont conçues pour accompagner

les entreprises dans leur croissance tout en restant adaptables à leurs besoins spécifiques.

1.1.2 Domaines d'activité

SO SOLUTION propose une gamme de solutions couvrant plusieurs fonctions et secteurs d'activité. Parmi les principaux domaines proposés par l'entreprise figurent :

- **Comptabilité et Finance** : solutions permettant d'automatiser la tenue comptable et la gestion financière, tout en prenant en compte les exigences de la réglementation marocaine, notamment en matière de TVA, de FEC et de déclarations fiscales.
- **Paie et Ressources Humaines** : solutions dédiées à la gestion des collaborateurs, des bulletins de paie et des déclarations sociales, adaptées au contexte réglementaire marocain.
- **Gestion commerciale** : solutions permettant de gérer le cycle commercial de l'entreprise, depuis les achats et les ventes jusqu'à la facturation, la gestion des stocks et le suivi des flux.
- **ERP** : une solution intégrée permettant de centraliser les différents processus de l'entreprise et de disposer d'une vision globale de son activité.
- **Promotion immobilière** : une solution spécialisée destinée aux promoteurs immobiliers marocains, couvrant notamment la gestion foncière, technique, commerciale, financière et juridique.
- **BTP** : une solution dédiée aux entreprises du bâtiment et des travaux publics, permettant notamment le suivi des chantiers, des achats, des stocks, de la sous-traitance et du contrôle financier.

1.1.3 Fiche technique

Raison sociale	SO SOLUTION
Siège social	74 Rue Ziri Ibnou Attia, Casablanca 20250, Maroc
Secteur d'activité	Édition de logiciels et solutions digitales de gestion
Expérience	30 ans d'expérience
Principales solutions	Comptabilité et Finance, Paie et RH, ERP, Gestion commerciale, BTP et Promotion immobilière
Site Web	https://somaroc.com/
Email	contact@somaroc.com
Téléphone	+212 5 22 61 19 45

TABLE 1.1 – Fiche technique de SO SOLUTION

1.1.4 L'organigramme de SO SOLUTION

L'organisation interne de SO SOLUTION s'articule autour de plusieurs fonctions permettant d'assurer le développement, la commercialisation, le déploiement et le support de ses solutions digitales. L'entreprise mobilise ainsi différentes compétences liées notamment au développement logiciel, à la gestion de projets, à l'accompagnement des clients et au support technique.

[À COMPLETER - description de l'organisation de SO Solutions]

1.2 Présentation du projet

1.2.1 Problématique

La gestion des factures fournisseurs reste, dans de nombreuses entreprises, un processus largement manuel : saisie répétée des montants, classification approximative des

dépenses, et absence de vision consolidée sur les coûts par fournisseur ou par catégorie. Cette approche est chronophage et propice aux erreurs de saisie.

Les progrès récents des modèles d'intelligence artificielle multimodaux offrent aujourd'hui la possibilité d'analyser directement des documents (images ou PDF) et d'en extraire des informations structurées, ouvrant la voie à une automatisation fiable de ce processus. Le défi consiste alors à concevoir une solution qui allie fiabilité de l'extraction, simplicité d'utilisation, et adaptabilité aux besoins spécifiques de chaque entreprise en matière de catégorisation des dépenses.

1.2.2 Solutions proposées

Pour notre projet, nous avons proposé les solutions suivantes :

- **Extraction et classification automatiques** : une extraction et une classification automatiques des factures en un seul appel à un modèle d'intelligence artificielle multimodal, permettant d'obtenir le fournisseur, la date, les montants HT/TVA/TTC et le numéro de facture directement à partir d'une image ou d'un PDF.
- **Vérification humaine** : une étape de vérification humaine systématique avant tout enregistrement définitif, afin que l'utilisateur garde le contrôle sur les données extraites et puisse corriger toute erreur d'extraction.
- **Catégories personnalisables** : un système de catégories de dépenses entièrement personnalisables par l'utilisateur, définies manuellement ou importées en masse via un fichier CSV, plutôt qu'une liste fixe de catégories imposée par l'application.
- **Tableau de bord et rapports** : un tableau de bord de suivi des dépenses par catégorie et par fournisseur, ainsi qu'un export de rapports au format Excel, afin de fournir une vision consolidée et exploitable des dépenses.

1.3 Planification du projet

1.3.1 Méthodologie de travail : Agile Scrum

Afin d’assurer une organisation efficace et un suivi régulier de l’avancement du projet, nous avons adopté une approche basée sur la méthodologie **Agile**, plus précisément sur le framework **Scrum**.

Le projet a été réalisé de manière itérative, en organisant le travail en plusieurs étapes et en adaptant progressivement les priorités selon les résultats obtenus. Cette approche nous a notamment permis de consacrer une période importante au benchmarking des modèles d’intelligence artificielle multimodaux avant de sélectionner le modèle le plus adapté aux besoins du projet.

Le développement a ensuite été réalisé progressivement, avec des phases de conception, d’implémentation, de tests et de corrections. Cette organisation a facilité le suivi de l’avancement et l’adaptation de la solution au fur et à mesure de son développement.

1.3.2 Diagramme de Gantt

Notre projet est géré en suivant un découpage chronologique en phases, qui spécifie ce qui doit être fait et sur quelle période. Pour représenter cette répartition, nous utilisons un diagramme de Gantt, un type de graphique à barres qui illustre les différentes étapes du projet au cours du temps.

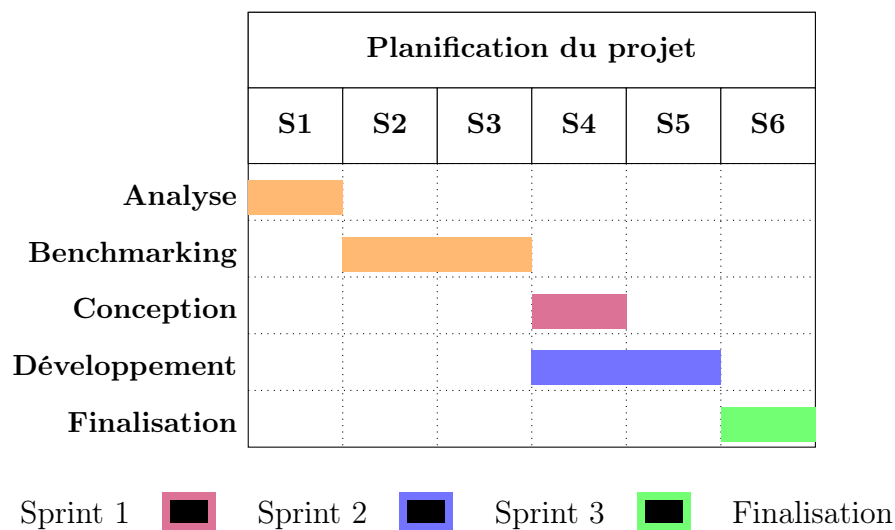


FIGURE 1.1 – Diagramme de Gantt du projet selon une approche Agile Scrum

1.4 Équipe de travail

Ce projet a été réalisé par :

— **AMERKAD Soumia** : Élève ingénieur en génie logiciel à l'ENSIAS.

1.5 Conclusion

Ce premier chapitre a présenté le contexte général du projet ainsi que l'environnement de travail au sein de SO Solutions. Nous avons également exposé la problématique liée à la gestion des factures fournisseurs et les principales solutions proposées.

Le chapitre suivant sera consacré à l'étude des besoins et des exigences de l'application.

Chapitre 2

Analyse et Conception

2.1 Introduction

Après avoir présenté le contexte général du projet, ce chapitre est consacré à l'analyse des besoins et à la conception de la solution proposée.

L'objectif de cette phase est d'identifier les fonctionnalités attendues, les contraintes auxquelles l'application doit répondre ainsi que les différents acteurs qui interagissent avec le système.

La phase de conception permet ensuite de définir l'architecture générale de l'application, la structure des données ainsi que le fonctionnement des principaux processus, notamment le traitement automatique des factures, l'authentification et la gestion des catégories.

2.2 Analyse des besoins

L'analyse des besoins permet de définir les fonctionnalités que le système doit fournir ainsi que les exigences de qualité auxquelles il doit répondre. Elle permet également d'identifier les différents acteurs intervenant dans le fonctionnement de l'application.

2.2.1 Besoins fonctionnels

Les besoins fonctionnels décrivent les principales fonctionnalités que l'application doit fournir aux différents utilisateurs.

Gestion des comptes

L'application doit permettre à un utilisateur de créer et de gérer son compte. Les principales fonctionnalités sont les suivantes :

- créer un compte ;
- se connecter à son compte ;
- accéder uniquement aux données qui lui sont associées ;
- se déconnecter de l'application.

Le système distingue deux rôles principaux : **Utilisateur** et **Administrateur**. Les droits d'accès aux différentes fonctionnalités dépendent du rôle associé au compte.

Gestion des catégories

L'utilisateur doit pouvoir personnaliser les catégories utilisées pour classifier ses dépenses.

Le système doit permettre :

- l'ajout manuel d'une catégorie ;
- la suppression d'une catégorie ;
- l'importation de plusieurs catégories à partir d'un fichier CSV ;
- l'utilisation de ces catégories lors de la classification automatique des factures.

Chaque utilisateur dispose de ses propres catégories afin de permettre une personnalisation de la classification des dépenses.

Gestion et analyse des factures

L'application doit permettre à l'utilisateur de téléverser une facture sous différents formats, notamment PDF, JPG ou PNG.

Après le téléversement, le système transmet le document au service d'intelligence artificielle afin d'en extraire automatiquement les principales informations.

Les informations recherchées sont notamment :

- le nom du fournisseur ;
- la date de la facture ;
- le numéro de facture ;
- le montant hors taxes (HT) ;
- le montant de la TVA ;
- le montant toutes taxes comprises (TTC) ;
- la catégorie de dépense.

Les informations extraites sont ensuite retournées à l'application et présentées à l'utilisateur. Celui-ci peut vérifier et corriger les données avant leur enregistrement définitif.

Consultation et gestion des factures

L'utilisateur doit pouvoir consulter les factures déjà enregistrées et effectuer différentes opérations de gestion.

Le système doit notamment permettre :

- consulter la liste des factures ;
- rechercher une facture ;
- filtrer les factures par fournisseur ;
- filtrer les factures par catégorie ;
- filtrer les factures par mois ;
- modifier les informations d'une facture ;
- supprimer une facture.

L'accès aux factures est limité aux données appartenant à l'utilisateur authentifié.

Tableau de bord et rapports

L'application doit fournir un tableau de bord permettant à l'utilisateur d'obtenir une vision synthétique de ses dépenses.

Le tableau de bord doit notamment présenter :

- le montant total des dépenses ;
- le nombre de factures enregistrées ;
- la répartition des dépenses par catégorie ;
- la répartition des dépenses par fournisseur ;
- l'évolution mensuelle des dépenses.

L'utilisateur doit également pouvoir exporter les données sous forme de rapports Excel selon les critères sélectionnés, notamment le fournisseur et la période.

Gestion des utilisateurs par l'administrateur

L'application prévoit un rôle **Administrateur** disposant de fonctionnalités spécifiques de supervision de la plateforme.

L'administrateur doit pouvoir :

- consulter les statistiques globales de la plateforme ;
- consulter le nombre total d'utilisateurs ;
- consulter le nombre total de factures enregistrées ;
- consulter une estimation du coût d'utilisation du service d'intelligence artificielle ;
- consulter la liste des utilisateurs inscrits ;
- supprimer un compte utilisateur lorsque cela est nécessaire.

Ces fonctionnalités sont exclusivement accessibles aux comptes disposant du rôle Administrateur.

2.2.2 Besoins non fonctionnels

Les besoins non fonctionnels définissent les contraintes de qualité que l'application doit respecter afin d'assurer son bon fonctionnement, sa sécurité et sa facilité d'utilisation.

Sécurité

La sécurité constitue une exigence importante de l'application.

Chaque utilisateur doit pouvoir accéder uniquement aux factures et aux catégories qui lui sont associées. L'API doit protéger les ressources nécessitant une authentification et vérifier l'identité ainsi que le rôle de l'utilisateur avant d'autoriser l'accès.

Le système distingue au minimum deux rôles :

- **Utilisateur** : accès à ses propres données et fonctionnalités ;
- **Administrateur** : accès aux fonctionnalités de supervision et de gestion des comptes.

Cette séparation des rôles permet de limiter l'accès aux ressources sensibles et d'assurer l'isolation des données entre les différents utilisateurs.

Performance

Le traitement automatique des factures doit être réalisé dans un délai raisonnable.

L'architecture doit notamment limiter les échanges inutiles avec le service d'intelligence artificielle afin de réduire le temps de traitement et la consommation des ressources externes.

Fiabilité

Les informations extraites automatiquement doivent pouvoir être vérifiées par l'utilisateur avant leur enregistrement définitif.

Cette validation humaine permet de limiter l'impact des éventuelles erreurs d'extraction ou de classification produites lors du traitement automatique.

Ergonomie

L'application doit proposer une interface simple, intuitive et responsive.

Les différentes fonctionnalités doivent être facilement accessibles afin de permettre à l'utilisateur de téléverser, vérifier et consulter ses factures sans complexité inutile.

Maintenabilité et évolutivité

L'application doit être conçue de manière modulaire afin de faciliter sa maintenance et son évolution.

La séparation des différentes responsabilités entre les composants permet notamment de faciliter l'ajout de nouvelles fonctionnalités et l'évolution du système.

Les catégories de dépenses doivent également être configurables par l'utilisateur sans nécessiter de modification du code source.

2.2.3 Acteurs du système

Le système comporte principalement trois acteurs :

- **Utilisateur** : il interagit directement avec l'application pour gérer ses catégories, téléverser et vérifier ses factures, consulter ses dépenses et générer des rapports.

- **Administrateur** : il assure la supervision de la plateforme. Il peut consulter les statistiques globales, consulter la liste des utilisateurs et gérer les comptes utilisateurs.
- **Service d'intelligence artificielle** : il intervient lors de l'analyse des factures afin d'extraire les informations pertinentes et de proposer une catégorie de dépense.

2.2.4 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation permet de représenter les principales fonctionnalités du système ainsi que les interactions entre les acteurs et l'application.

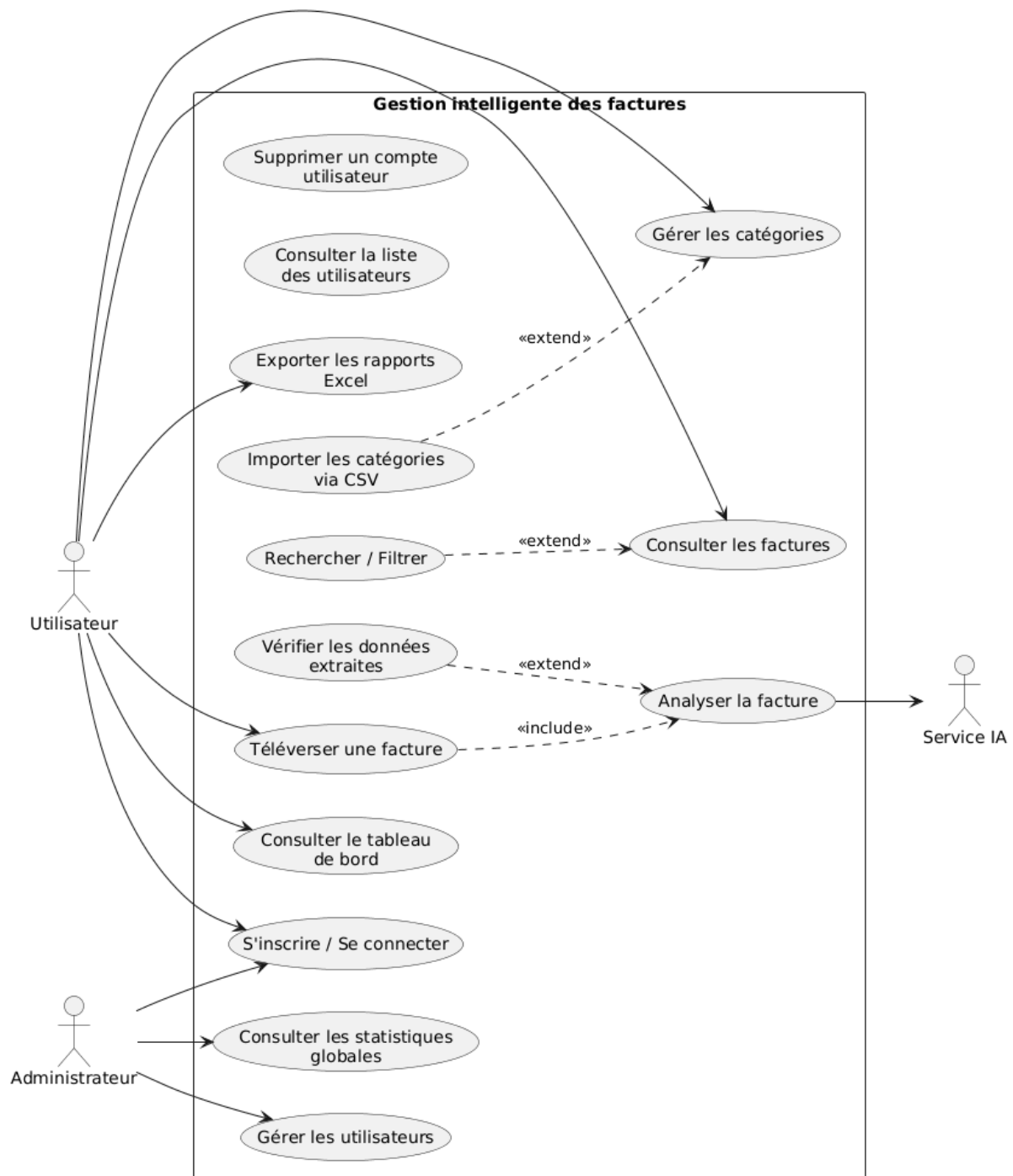


FIGURE 2.1 – Diagramme de cas d'utilisation de l'application

Le diagramme met en évidence les principales interactions entre les acteurs et la plateforme.

L'utilisateur peut notamment s'authentifier, gérer ses catégories, téléverser et vérifier ses factures, consulter ses données, accéder au tableau de bord et exporter des rapports.

L'administrateur dispose de fonctionnalités spécifiques liées à la supervision de la plateforme. Il peut consulter les statistiques globales et gérer les comptes utilisateurs.

Le service d'intelligence artificielle intervient dans le processus d'analyse des factures. Il est sollicité après le téléversement d'une facture afin d'en analyser le contenu et d'extraire automatiquement les informations nécessaires.

2.3 Conception

Cette section présente la conception de la solution à partir des besoins identifiés précédemment. Elle décrit l'architecture générale de l'application, la structure des données ainsi que la conception des principaux processus fonctionnels.

2.3.1 Architecture générale

L'application adopte une architecture client-serveur organisée autour de plusieurs composants principaux : le frontend, le backend, le service d'intelligence artificielle et la base de données.

Le **frontend** constitue l'interface utilisateur. Il permet notamment de téléverser les factures, de consulter les données extraites, de gérer les catégories et de visualiser les dépenses.

Le **backend** constitue le cœur de l'application. Il assure la gestion des utilisateurs, des catégories et des factures. Il est également responsable de la communication avec la base de données et avec le service d'intelligence artificielle.

Le **service d'intelligence artificielle** intervient lors de l'analyse des factures. Le backend lui transmet les documents ainsi que les informations nécessaires au traitement. Le résultat obtenu est ensuite retourné au backend afin d'être présenté à l'utilisateur.

Enfin, la **base de données** assure la persistance des informations relatives aux utilisateurs, aux catégories et aux factures.

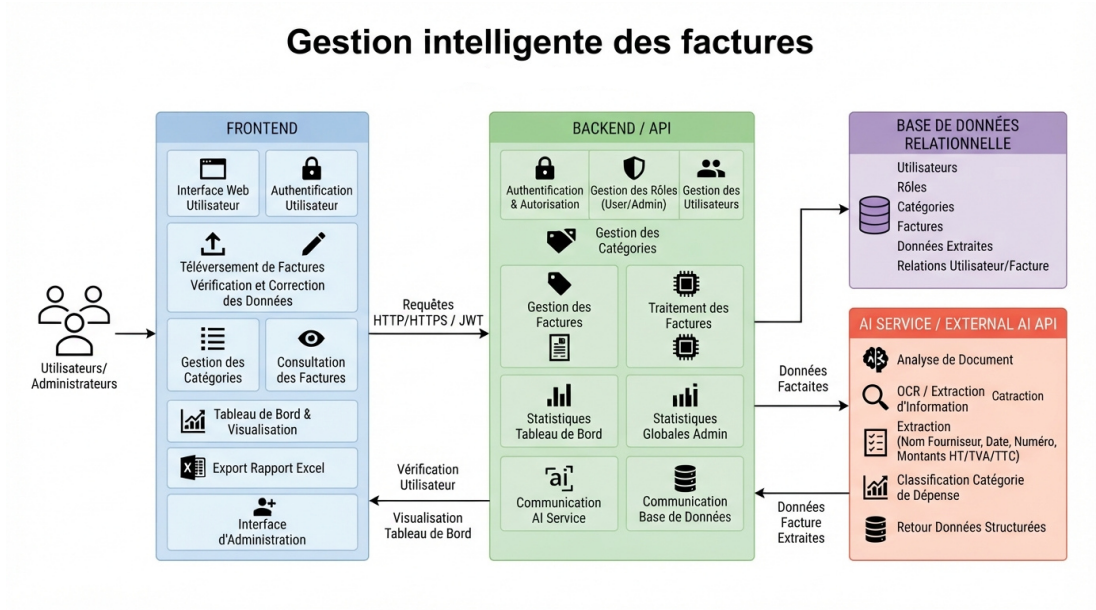


FIGURE 2.2 – Architecture générale de l’application

2.3.2 Modélisation du système

Diagramme de classes

La modélisation du système permet de représenter sa structure statique à travers les principales classes métier ainsi que les relations qui existent entre elles.

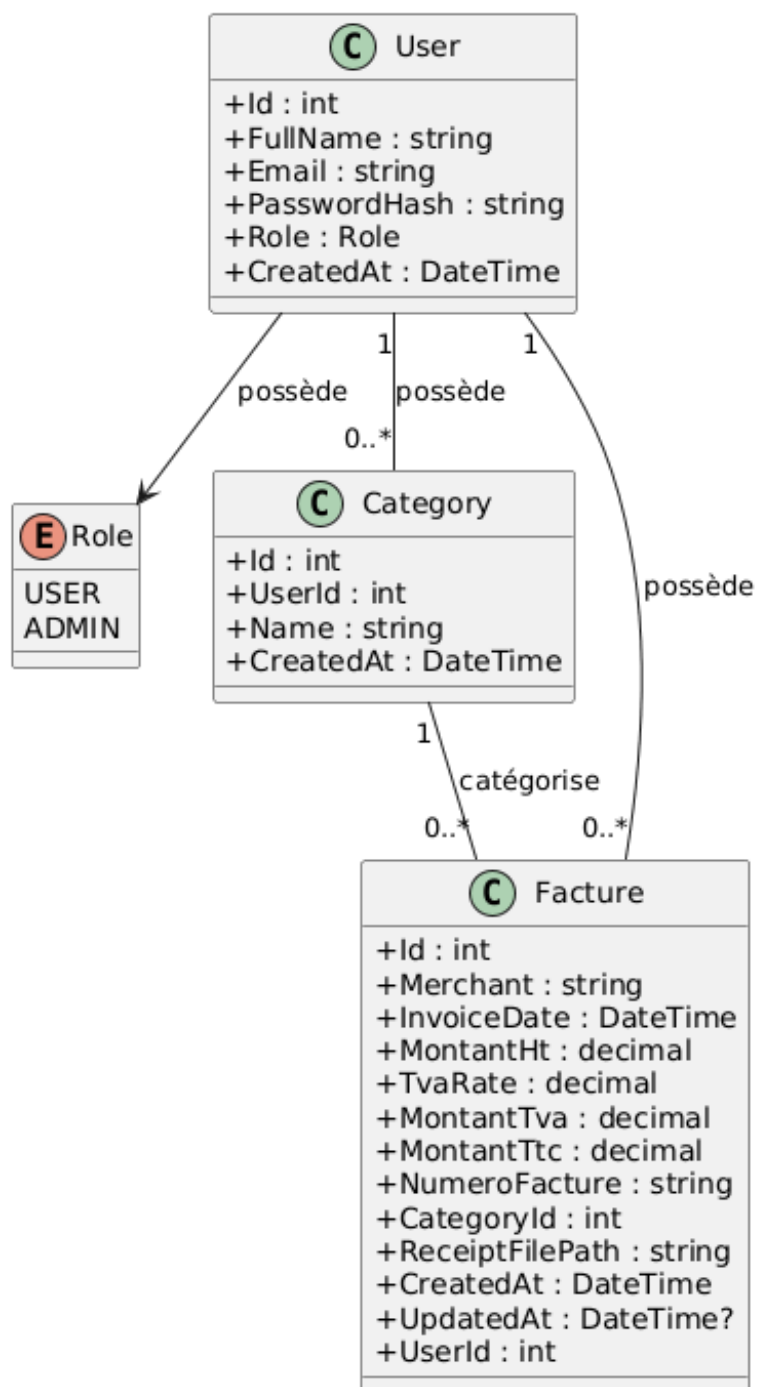
Diagramme de classes de InvoiceAI

FIGURE 2.3 – Diagramme de classes de l'application

Description des entités

Le diagramme de classes présente les principales entités intervenant dans le fonctionnement de l'application InvoiceAI ainsi que les relations qui les associent.

User : représente un utilisateur de l'application. Il contient les informations nécessaires à son identification et à la gestion de son compte, notamment son nom complet (*FullName*), son adresse e-mail (*Email*), son mot de passe sous forme de hash (*PasswordHash*), son rôle (*Role*) ainsi que sa date de création. Un utilisateur peut posséder plusieurs catégories et plusieurs factures.

Role : est une énumération permettant de définir le niveau d'accès d'un utilisateur. Deux rôles sont prévus dans l'application : *USER*, correspondant à un utilisateur standard, et *ADMIN*, correspondant à un administrateur.

Category : représente une catégorie personnalisée permettant à un utilisateur de classer ses factures. Elle contient un identifiant (*Id*), l'identifiant de l'utilisateur propriétaire (*UserId*), le nom de la catégorie (*Name*) ainsi que sa date de création (*CreatedAt*). Un utilisateur peut créer plusieurs catégories et une catégorie peut être associée à plusieurs factures.

Facture : représente une facture enregistrée dans l'application. Elle regroupe les informations extraites ou saisies concernant la facture, notamment le commerçant (*Merchant*), la date de facturation (*InvoiceDate*), les montants hors taxes, de TVA et toutes taxes comprises (*MontantHt*, *MontantTva* et *MontantTtc*), le taux de TVA (*TvaRate*) et le numéro de facture (*NumeroFacture*). Elle contient également le chemin du fichier original (*ReceiptFilePath*), les dates de création et de modification, ainsi que les identifiants de l'utilisateur et de la catégorie associés.

Les relations indiquées dans le diagramme montrent qu'un utilisateur peut posséder plusieurs catégories et plusieurs factures. Chaque catégorie peut, à son tour, être associée à plusieurs factures. Le rôle est associé à l'utilisateur afin de déterminer ses droits d'accès au sein de l'application.

2.3.3 Étude comparative des plateformes d’intelligence artificielle

Introduction et objectif de l’étude

Afin de sélectionner la solution d’intelligence artificielle la plus adaptée aux besoins de l’application, une étude comparative de plusieurs plateformes et solutions d’IA a été réalisée. Cette étude porte sur des solutions cloud et locales, notamment Google AI, Amazon Bedrock, Azure AI, OpenAI et Ollama.

La comparaison s’appuie sur plusieurs critères liés au traitement des documents, notamment la compréhension documentaire, les capacités de vision et d’OCR, l’extraction structurée des informations, la précision de la classification, les possibilités d’intégration via API, ainsi que la sécurité et le coût d’utilisation. L’objectif est d’identifier les avantages et les limites de chaque solution afin de justifier le choix de la plateforme et du modèle utilisés dans l’application.

Critères d’évaluation

Les plateformes sont comparées selon les critères suivants :

TABLE 2.1 – Critères d’évaluation des solutions d’intelligence artificielle

Critère	Description
Compréhension documentaire	Capacité à analyser et comprendre le contenu d’un document.
Vision	Capacité à traiter directement des documents ou images.
OCR	Capacité à identifier et extraire le texte présent dans des documents numérisés.
Extraction structurée	Capacité à retourner les informations extraites dans une structure exploitable par l’application.
Classification	Capacité à identifier la catégorie ou le type d’un document.
API	Disponibilité d’interfaces permettant l’intégration avec l’application.
Facilité d’intégration	Complexité de mise en œuvre et d’intégration dans l’architecture existante.
Sécurité	Mécanismes proposés pour protéger les données et les échanges.
Offre gratuite	Disponibilité d’un niveau gratuit ou d’une période d’essai.
Coût	Coût d’utilisation des services ou des modèles.

Comparaison des plateformes

Les plateformes retenues sont comparées selon les critères définis précédemment. L'analyse prend en compte leurs capacités de traitement documentaire, leurs possibilités d'intégration, ainsi que leurs modalités d'utilisation et de tarification.

TABLE 2.2 – Comparaison des plateformes d'intelligence artificielle selon les critères retenus

Critère	Google AI	Amazon Be-drock	Azure AI	OpenAI	Ollama
Compréhension documentaire	Très bonne	Très bonne	Très bonne	Très bonne	Dépend du modèle
Vision	Oui	Selon le modèle	Oui	Oui	Oui pour les modèles compatibles
OCR	Intégré aux modèles multimodaux	Possible avec les modèles compatibles et Textract	Document Intelligence	Possible avec les modèles multimodaux	Possible avec les modèles de vision
Extraction structurée	Oui	Oui	Oui	Oui	Oui, selon le modèle
Classification	Très bonne	Très bonne	Très bonne	Très bonne	Dépend du modèle
API	API disponible	API AWS	API Azure	API disponible	API locale
Facilité d'intégration	Élevée	Moyenne	Moyenne	Élevée	Élevée en environnement local
Sécurité	Service cloud	Intégration AWS et contrôles IAM	Intégration Azure et contrôles d'accès	Service cloud	Exécution locale des modèles
Offre gratuite	Oui, selon les modèles et conditions	Selon les services et offres AWS	Crédit ou offre d'essai selon les conditions	Non	Oui, logiciel local
Coût	Facturation selon les tokens	Facturation selon les modèles et services	Facturation selon les services utilisés	Facturation selon les tokens	\$0 d'API, mais coût matériel

Analyse des plateformes

Google AI

Google AI propose la famille de modèles Gemini, capables de traiter directement du texte, des images et des documents. Cette approche multimodale permet d'effectuer au sein d'une même chaîne de traitement la compréhension du document, l'extraction des informations et leur classification. La plateforme propose également une API permettant une intégration directe dans l'application, ce qui limite le nombre de services à gérer et simplifie l'architecture globale.

Amazon Bedrock

Amazon Bedrock donne accès à plusieurs familles de modèles d'intelligence artificielle, notamment Claude, Amazon Nova, Llama, Mistral et Cohere. Les capacités de traitement des documents et des images dépendent du modèle sélectionné. Pour les besoins nécessitant une extraction documentaire spécialisée, notamment l'OCR, les formulaires ou les tableaux, Bedrock peut être associé à Amazon Textract. Cette architecture offre une grande flexibilité et permet de combiner plusieurs services AWS, mais elle peut également augmenter la complexité de mise en œuvre.

Microsoft Azure AI

Azure AI propose plusieurs services adaptés au traitement documentaire. Azure AI Document Intelligence permet notamment d'extraire des informations à partir de documents, tandis que les modèles d'intelligence artificielle peuvent être utilisés pour les tâches de compréhension, d'extraction et de classification. Cette séparation entre les services spécialisés offre une grande flexibilité et des possibilités avancées d'intégration dans l'écosystème Azure, mais peut nécessiter la configuration de plusieurs composants.

OpenAI Platform

OpenAI Platform met à disposition des modèles multimodaux capables de traiter du texte et des images et pouvant être utilisés pour l'extraction d'informations structurées et la classification de documents. Son API permet une intégration directe dans une application et offre différents mécanismes pour structurer les réponses des modèles. La tarification repose sur la consommation, ce qui nécessite de prendre en compte le volume de documents traités lors de l'évaluation du coût.

Ollama

Ollama permet d'exécuter localement différents modèles de langage, notamment certains modèles disposant de capacités de vision. Contrairement à une approche exclusive-

ment basée sur des services cloud, l'exécution locale permet de conserver les documents sur la machine utilisée et d'éviter les coûts liés à l'appel d'une API externe.

Cependant, les capacités de traitement documentaire dépendent fortement du modèle choisi. Par exemple, certains modèles compatibles avec Ollama, tels que Llama 3.2 Vision, peuvent recevoir des images et effectuer des tâches d'analyse visuelle, notamment l'extraction de texte. Cette approche nécessite toutefois des ressources matérielles suffisantes et peut être plus contraignante pour obtenir des performances adaptées au traitement régulier de documents.

Remarques

L'étude comparative met en évidence plusieurs différences entre les solutions étudiées :

- Les plateformes cloud telles que Google AI, Amazon Bedrock, Azure AI et OpenAI proposent des modèles multimodaux permettant le traitement de documents et d'images, avec des niveaux d'intégration et des architectures différents.
- Amazon Bedrock et Azure AI proposent une approche modulaire permettant d'associer des modèles d'intelligence artificielle à des services spécialisés de traitement documentaire. Cette flexibilité peut cependant entraîner une architecture plus complexe.
- Ollama présente l'avantage d'une exécution locale et de l'absence de coût d'utilisation d'une API externe. Toutefois, ses performances et ses capacités de traitement documentaire dépendent du modèle choisi ainsi que des ressources matérielles disponibles.
- Le choix d'une plateforme ne dépend donc pas uniquement de ses capacités d'intelligence artificielle. La facilité d'intégration, la gestion des documents, les contraintes matérielles, la sécurité et le coût de traitement constituent également des critères importants.
- Google AI présente une approche intégrée particulièrement intéressante pour le projet, puisqu'un modèle multimodal peut être utilisé pour la compréhension du document, l'extraction des informations et leur classification sans nécessiter plu-

sieurs services spécialisés.

2.3.4 Comparaison des modèles Gemini

Après avoir comparé les différentes plateformes d’intelligence artificielle, une étude plus approfondie a été menée sur plusieurs variantes de la famille Gemini proposées par Google AI. L’objectif est d’identifier le modèle le plus adapté au traitement automatique des factures dans l’application.

Les modèles ont été évalués selon plusieurs critères : la qualité de l’extraction des informations comptables, la qualité de la classification des factures, la disponibilité du modèle via l’API et le coût estimé d’un traitement. Afin d’assurer une comparaison cohérente, les expérimentations ont été réalisées dans les mêmes conditions, en utilisant la même facture, le même prompt et le même format de sortie JSON.

Résultats des expérimentations Quatre variantes de Gemini ont été considérées : Gemini 2.5 Flash, Gemini 2.5 Flash-Lite, Gemini 3.1 Flash-Lite et Gemini 3.5 Flash.

Les résultats obtenus sont présentés dans le tableau suivant.

TABLE 2.3 – Résultats des expérimentations sur les modèles Gemini

Modèle	Extraction	Classification	Disponibilité
Gemini 2.5 Flash	Correcte	Correcte	Oui
Gemini 2.5 Flash-Lite	–	–	Non (erreur 404)
Gemini 3.1 Flash-Lite	Correcte	Correcte	Oui
Gemini 3.5 Flash	Correcte	Correcte	Oui

Les expérimentations montrent que Gemini 2.5 Flash, Gemini 3.1 Flash-Lite et Gemini 3.5 Flash ont produit des résultats corrects pour l’extraction des informations comptables ainsi que pour la classification des factures testées.

Le modèle Gemini 2.5 Flash-Lite n’a pas pu être évalué, car il avait été retiré de l’API au moment des expérimentations. Les appels effectués vers ce modèle retournaient une erreur HTTP 404 (NOT_FOUND). Il n’a donc pas été possible de mesurer ses performances ni d’estimer son coût dans les conditions expérimentales du projet.

Comparaison du coût de traitement

La comparaison des modèles est complétée par une estimation du coût d'un traitement. Pour garantir une comparaison équitable, les calculs utilisent les valeurs de consommation mesurées expérimentalement avec une facture, soit 980 tokens d'entrée et 900 tokens de sortie.

Le coût d'un traitement est calculé selon la formule suivante :

$$\text{Coût}_{total} = \frac{N_{entre}}{1\,000\,000}P_{entre} + \frac{N_{sortie}}{1\,000\,000}P_{sortie}$$

où N_{entre} et N_{sortie} représentent respectivement le nombre de tokens consommés en entrée et en sortie, tandis que P_{entre} et P_{sortie} correspondent aux tarifs appliqués par million de tokens.

Protocole expérimental

Afin de comparer les modèles dans des conditions similaires, le même document de facture a été utilisé pour les différentes expérimentations. Le prompt ainsi que la structure des données demandées ont également été conservés.

Le prompt demande au modèle d'extraire les principales informations de la facture, notamment :

- le nom du fournisseur ;
- la date de la facture ;
- le numéro de facture ;
- le montant hors taxes (HT) ;
- le montant de la TVA ;
- le montant toutes taxes comprises (TTC) ;
- la catégorie correspondant à la facture parmi les catégories définies par l'utilisateur.

Les informations financières sont extraites à partir du bloc récapitulatif des totaux de la facture et non à partir du détail des lignes. Les résultats sont retournés dans un format JSON afin de faciliter leur intégration dans l'application.

Mesure expérimentale

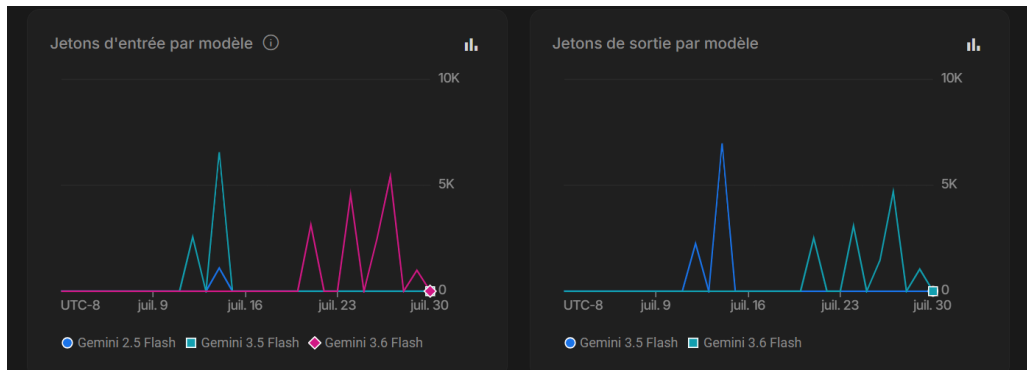


FIGURE 2.4 – Évolution de la consommation des tokens observée lors des expérimentations.

Lors du test réalisé avec le modèle *Gemini 3.5 Flash*, la requête a consommé environ 980 tokens en entrée et 900 tokens en sortie.

En utilisant les tarifs retenus pour cette expérimentation, soit 1,50 \$ par million de tokens en entrée et 9,00 \$ par million de tokens en sortie, le coût peut être calculé comme suit :

$$\text{Coût}_{\text{entrée}} = \frac{980}{1\,000\,000} \times 1,50 = 0,00147 \$$$

$$\text{Coût}_{\text{sortie}} = \frac{900}{1\,000\,000} \times 9,00 = 0,00810 \$$$

Ainsi :

$$\text{Coût}_{\text{total}} = 0,00147 + 0,00810 = 0,00957 \$$$

Le coût estimé pour le traitement d'une facture est donc d'environ **0,01 \$ par facture** dans les conditions de cette expérimentation.

Les estimations obtenues pour les modèles disponibles sont présentées dans le tableau suivant.

TABLE 2.4 – Estimation du coût de traitement d’une facture selon le modèle

Modèle	Prix entrée (\$/M tokens)	Prix sortie (\$/M tokens)	Coût par facture (\$)
Gemini 2.5 Flash	0.30	2.50	0.00254
Gemini 3.1 Flash-Lite	0.25	1.50	0.00160
Gemini 3.5 Flash	1.50	9.00	0.00957

Les résultats montrent que Gemini 3.1 Flash-Lite présente le coût théorique le plus faible parmi les modèles disponibles et évalués, avec un coût estimé de 0,00160 \$ par facture. Gemini 2.5 Flash présente également un coût inférieur à celui de Gemini 3.5 Flash, avec une estimation de 0,00254 \$ par facture.

En revanche, le coût estimé d’un traitement avec Gemini 3.5 Flash est de 0,00957 \$, soit environ 0,01 \$ par facture. Ce coût reste limité au regard des opérations réalisées, qui comprennent à la fois l’extraction des informations comptables et la classification automatique du document.

Choix du modèle Les expérimentations réalisées montrent que les modèles disponibles et accessibles ont permis d’obtenir des résultats corrects pour les tâches d’extraction et de classification considérées. Le coût constitue toutefois un critère différenciant entre les modèles.

Bien que Gemini 3.1 Flash-Lite présente le coût estimé le plus faible, Gemini 3.5 Flash a été retenu pour l’application en raison de sa disponibilité et de sa robustesse lors des expérimentations réalisées.

Le coût mesuré avec Gemini 3.5 Flash, d’environ 0,01 \$ par facture, reste compatible avec les objectifs du projet. Le modèle retenu permet ainsi de réaliser les opérations d’extraction et de classification nécessaires au traitement automatique des factures.

Choix de la solution

À l’issue de cette étude comparative, le choix de la solution est effectué en tenant compte des besoins spécifiques de l’application, notamment l’analyse de documents, l’ex-

traction des informations des factures, leur classification et la nécessité d'une intégration simple avec le système développé.

La solution retenue est ensuite évaluée expérimentalement sur des factures réelles afin de vérifier la qualité des informations extraites et d'estimer le coût associé au traitement d'une facture. Cette évaluation permet de compléter la comparaison théorique par des résultats obtenus dans le contexte réel de l'application.

2.3.5 Conception des principaux processus

Cette section présente le fonctionnement des principaux processus de l'application à travers des diagrammes de séquence.

Traitement d'une facture

Le traitement d'une facture implique plusieurs composants : l'utilisateur, le frontend, le backend, le service d'intelligence artificielle et la base de données.

Le processus suit les étapes suivantes :

1. L'utilisateur sélectionne et téléverse une facture.
2. Le frontend transmet le document au backend.
3. Le backend vérifie la validité du document.
4. Le backend transmet la facture au service d'intelligence artificielle.
5. Le service analyse le contenu du document.
6. Les informations comptables sont extraites et une catégorie est proposée.
7. Les résultats sont retournés au backend.
8. Le backend retourne les informations au frontend.
9. L'utilisateur vérifie et corrige éventuellement les informations.
10. Après validation, le backend enregistre la facture dans la base de données.

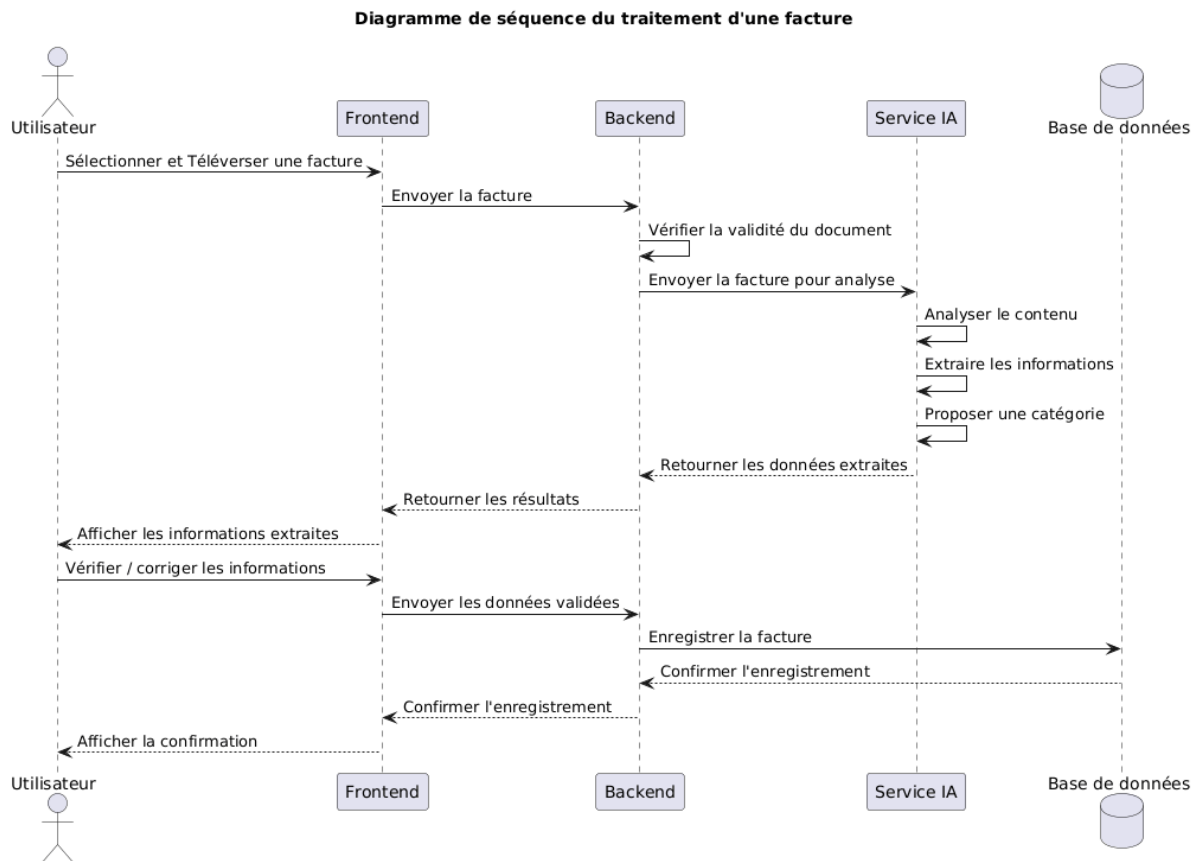


FIGURE 2.5 – Diagramme de séquence du traitement d'une facture

Cette approche permet de combiner l'automatisation de l'extraction avec une validation humaine avant l'enregistrement définitif des données.

Authentification

Le processus d'authentification permet à l'utilisateur d'accéder aux ressources protégées de l'application.

Le processus général est le suivant :

1. L'utilisateur saisit ses informations d'identification.
2. Le frontend transmet les informations au backend.
3. Le backend vérifie les informations fournies.
4. En cas de succès, un jeton d'authentification est généré.
5. Le jeton est retourné au frontend.

6. Le frontend utilise ce jeton pour accéder aux ressources protégées.

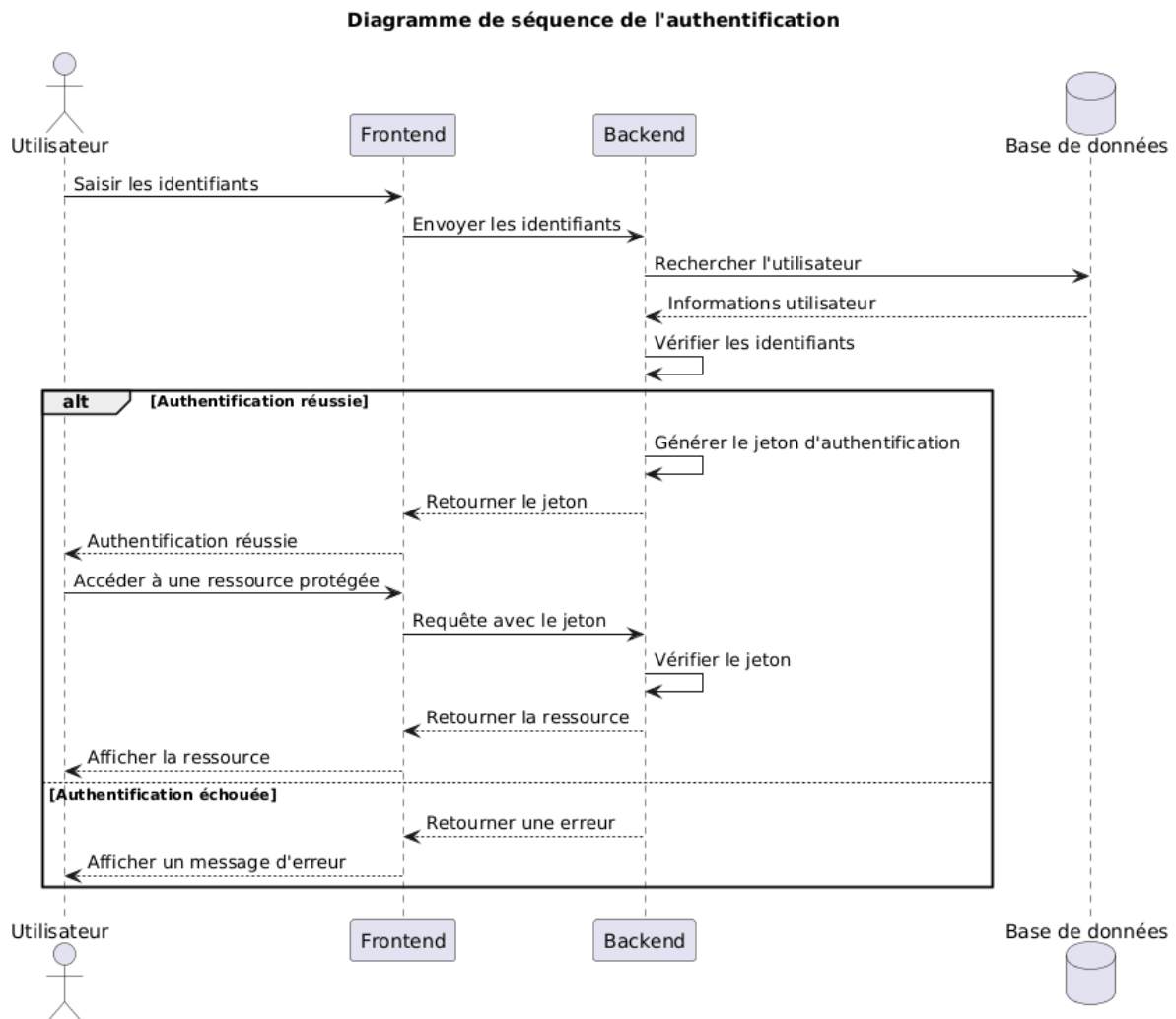


FIGURE 2.6 – Diagramme de séquence de l'authentification

L'authentification repose sur un mécanisme de jetons permettant de sécuriser l'accès aux ressources de l'application.

Lors de la connexion, les informations d'identification fournies par l'utilisateur sont vérifiées par le backend.

En cas de succès, un jeton d'authentification est généré et retourné au client.

Pour chaque requête nécessitant une authentification, le backend vérifie la présence et la validité du jeton afin d'identifier l'utilisateur associé à la requête.

Le rôle de l'utilisateur est également pris en compte afin de contrôler l'accès aux fonctionnalités réservées à l'administrateur.

Cette approche permet notamment de garantir que les données retournées par l'API correspondent à l'utilisateur authentifié.

Gestion des catégories

La gestion des catégories permet à l'utilisateur de personnaliser les catégories utilisées pour classer ses dépenses.

Deux méthodes de création sont disponibles :

- la création manuelle d'une catégorie ;
- l'importation de plusieurs catégories à partir d'un fichier CSV.

Pour l'ajout manuel d'une catégorie, le processus est le suivant :

1. L'utilisateur saisit le nom de la catégorie.
2. Le frontend transmet la demande au backend.
3. Le backend vérifie l'identité de l'utilisateur.
4. La nouvelle catégorie est enregistrée dans la base de données.
5. Le backend retourne le résultat au frontend.

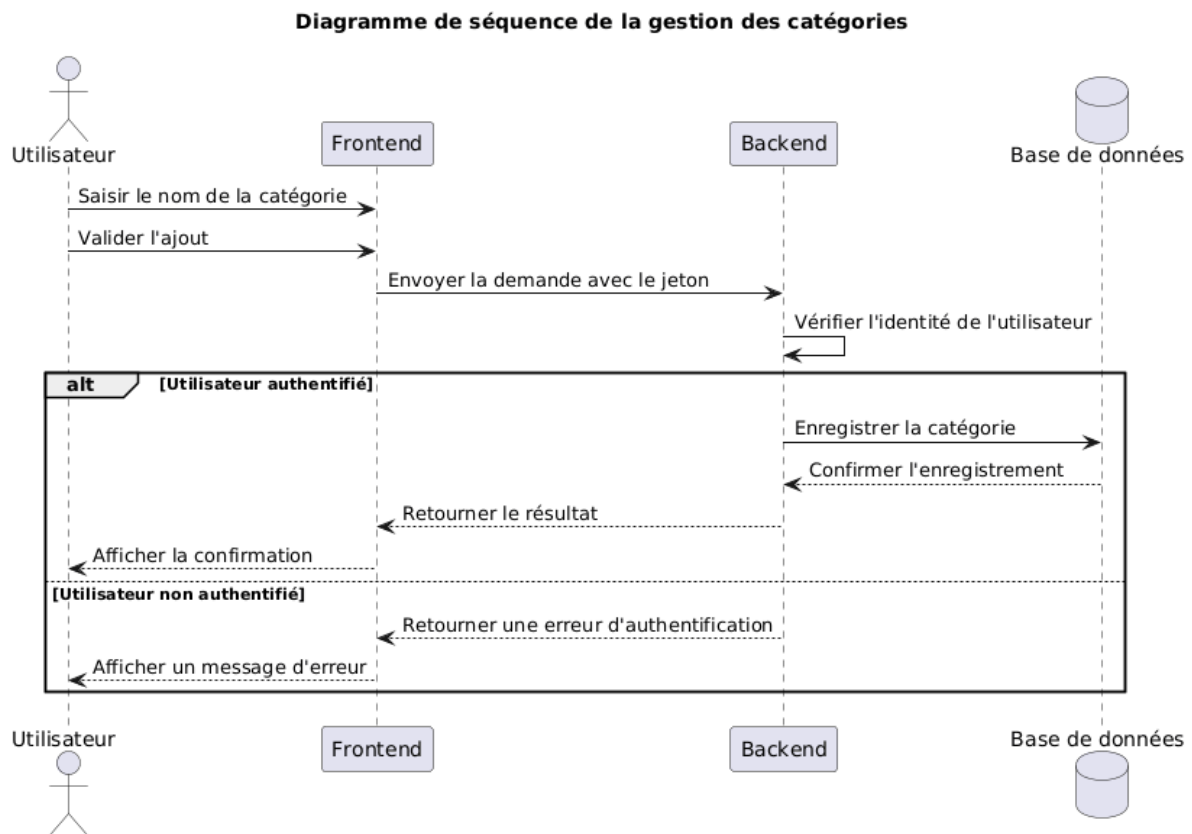


FIGURE 2.7 – Diagramme de séquence de la gestion des catégories

Le même principe est utilisé pour la suppression d’une catégorie et l’importation de plusieurs catégories à partir d’un fichier CSV.

2.4 Conclusion

Ce chapitre a permis de définir les besoins et de concevoir l’architecture, la structure et les principaux processus de l’application, ainsi que de sélectionner la solution d’intelligence artificielle adaptée. Le chapitre suivant présente la réalisation et l’implémentation de la solution.

Chapitre 3

Réalisation

3.1 Introduction

Ce chapitre présente la réalisation de l'application InvoiceAI. Il décrit les outils et technologies utilisés ainsi que l'implémentation des principales fonctionnalités et les interfaces réalisées.

3.2 Outils utilisés

3.2.1 Visual Studio Code



FIGURE 3.1 – Logo de Visual Studio Code

Visual Studio Code a été utilisé comme environnement de développement principal pour la réalisation du frontend et du backend. Il a permis d'éditer le code, d'effectuer le débogage, de gérer le projet avec Git et d'utiliser différentes extensions facilitant le développement.

3.2.2 SQL Server Management Studio



FIGURE 3.2 – Logo de SQL Server Management Studio

SQL Server Management Studio (SSMS) a été utilisé pour administrer la base de données SQL Server, consulter les données enregistrées et exécuter des requêtes permettant de vérifier le bon fonctionnement de la persistance des données.

3.2.3 Postman



FIGURE 3.3 – Logo de Postman

Postman a été utilisé pour tester les différents points d'accès de l'API backend indépendamment de l'interface frontend. Il a notamment permis de vérifier les opérations d'authentification, de gestion des catégories et de gestion des factures, ainsi que les réponses retournées par le serveur.

3.2.4 Google AI Studio



FIGURE 3.4 – Logo de Google AI Studio

Google AI Studio a été utilisé lors de la phase de développement pour expérimenter avec les modèles Gemini et affiner le prompt destiné à l'extraction des informations contenues dans les factures. Les essais ont permis de vérifier le format et la pertinence des données retournées avant l'intégration de l'API Gemini dans l'application.

3.3 Technologies utilisées

Les principales technologies utilisées pour développer InvoiceAI sont présentées dans le tableau 3.1.

Technologie	Rôle dans InvoiceAI
Next.js / TypeScript	Développement de l'interface utilisateur, gestion des routes et appels à l'API backend.
Tailwind CSS	Mise en forme des interfaces et définition de l'identité visuelle de l'application.
ASP.NET Core	Développement de l'API backend et implémentation de la logique métier.
Entity Framework Core	Mapping objet-relationnel entre l'application et la base de données.
SQL Server	Stockage des utilisateurs, catégories et factures.
Google Gemini API	Extraction et classification automatique des informations des factures.
JWT	Authentification et sécurisation des accès aux ressources de l'application.
ClosedXML	Génération des rapports d'export au format Excel.

TABLE 3.1 – Technologies utilisées dans InvoiceAI et leur rôle

3.4 Implémentation de la solution

Cette section présente la mise en œuvre des principales fonctionnalités de l'application, depuis l'authentification jusqu'au traitement et à l'exploitation des factures.

3.4.1 Implémentation de l'authentification

L'authentification a été implémentée au niveau du backend avec **ASP.NET Core** et des **JSON Web Tokens (JWT)**.

Lors de la connexion, les informations d'identification fournies par l'utilisateur sont vérifiées par le backend. Après validation, un token JWT est généré et transmis au frontend. Ce token est ensuite utilisé pour authentifier les requêtes nécessitant un accès protégé.

Le système distingue également les rôles *Utilisateur* et *Administrateur*. Les autorisations associées à chaque rôle permettent de contrôler l'accès aux différentes fonctionnalités de l'application.

3.4.2 Implémentation de la gestion des catégories

Chaque utilisateur peut définir ses propres catégories de dépenses. L'application permet d'ajouter et de supprimer des catégories directement depuis l'interface.

Une fonctionnalité d'importation permet également de charger plusieurs catégories à partir d'un fichier CSV. Ces catégories sont ensuite associées à l'utilisateur et utilisées lors de la classification automatique des factures.

La persistance des catégories est assurée par **Entity Framework Core** et **SQL Server**.

3.4.3 Implémentation du traitement automatique des factures

Le traitement d'une facture constitue l'une des principales fonctionnalités de l'application.

L'utilisateur commence par téléverser une facture au format PDF ou image. Le fichier est ensuite transmis au backend, qui prépare les données nécessaires à l'appel de l'API **Google Gemini**.

Un prompt structuré est envoyé au modèle afin d'extraire les principales informations de la facture, notamment :

- le nom du fournisseur ;
- le numéro de la facture ;
- la date ;
- le montant hors taxes (HT) ;
- le montant de la TVA ;
- le montant toutes taxes comprises (TTC) ;
- la catégorie de dépense.

La réponse du modèle est structurée afin de pouvoir être exploitée directement par le backend. Les données extraites sont ensuite retournées au frontend pour être vérifiées et éventuellement corrigées par l'utilisateur avant leur enregistrement définitif.

Cette étape de validation permet de conserver un contrôle humain sur les informations

extraites automatiquement.

3.4.4 Implémentation de la gestion des factures

Une fois les informations vérifiées, la facture est enregistrée dans la base de données. Chaque facture est associée à son utilisateur et à une catégorie.

L'application permet ensuite de consulter les factures enregistrées sous forme de liste. Des fonctionnalités de recherche et de filtrage permettent de retrouver une facture selon différents critères, tels que le fournisseur, la catégorie ou le mois.

L'utilisateur peut également modifier ou supprimer une facture existante. Les opérations sont protégées afin qu'un utilisateur ne puisse accéder qu'à ses propres données.

3.4.5 Implémentation du tableau de bord et des exports

Le tableau de bord permet de synthétiser les dépenses enregistrées dans l'application. Les données sont agrégées afin d'afficher notamment le montant total des dépenses, le nombre de factures, la répartition des dépenses par catégorie et leur évolution mensuelle.

Une analyse par fournisseur permet également de visualiser la répartition des dépenses selon les différents fournisseurs enregistrés.

Enfin, l'application permet de générer un rapport au format Excel à l'aide de la bibliothèque **ClosedXML**. Le rapport peut être généré selon une période et un fournisseur sélectionnés, facilitant ainsi l'exploitation des données en dehors de l'application.

3.5 Présentation de l'application

Cette section présente les principales interfaces développées pour InvoiceAI à travers différentes captures d'écran.

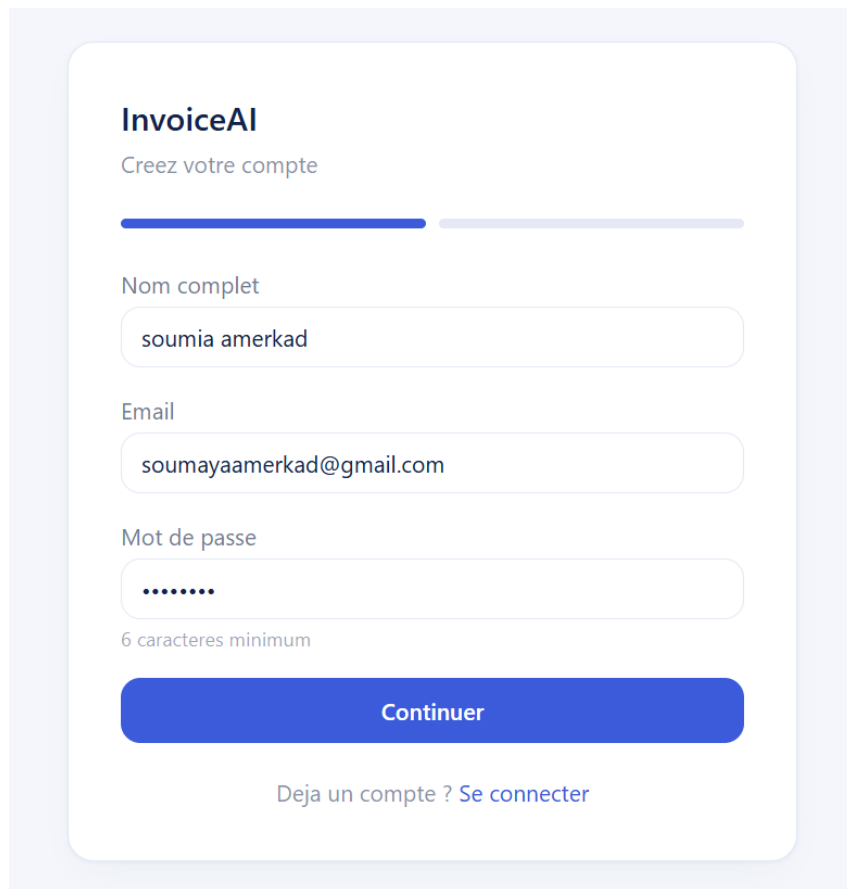
3.5.1 Page d'accueil

La page d'accueil constitue le point d'entrée public de l'application. Elle présente le principe général d'InvoiceAI ainsi que ses principales fonctionnalités avant l'authentification de l'utilisateur.

3.5.2 Inscription et définition des catégories

Lors de son inscription, l'utilisateur renseigne les informations nécessaires à la création de son compte. Il peut également définir les catégories de dépenses qu'il souhaite utiliser dans l'application.

Les catégories peuvent être ajoutées manuellement ou importées à partir d'un fichier CSV.



The image shows a web form for creating an account on InvoiceAI. The form is titled "InvoiceAI" and "Creez votre compte". It includes a progress bar, input fields for "Nom complet", "Email", and "Mot de passe", and a "Continuer" button. Below the password field, it states "6 caracteres minimum". At the bottom, there is a link "Deja un compte ? Se connecter".

InvoiceAI
Creez votre compte

Nom complet
soumia amerkad

Email
soumayaamerkad@gmail.com

Mot de passe
.....
6 caracteres minimum

Continuer

Deja un compte ? [Se connecter](#)



The screenshot shows a web interface for 'InvoiceAI'. At the top, the title 'InvoiceAI' is followed by the subtitle 'Définissez vos catégories de dépenses'. Below this is a horizontal blue bar. A text block explains: 'Tapez le nom d'une catégorie puis Entree pour l'ajouter, ou importez-les depuis un fichier CSV (une catégorie par ligne, ou séparées par virgules). Modifiable plus tard dans Paramètres.' Below the text is a text input field with the placeholder 'Ex: Matieres premieres, Transport...'. To the right of the input field are two links: '+ Ajouter cette catégorie' and 'Importer un CSV' with an upload icon. Below these links, it says 'Aucune catégorie ajoutée pour l'instant.' At the bottom, there are two buttons: 'Retour' and 'Créer mon compte'. Below the buttons, there is a link 'Déjà un compte ? Se connecter'.

FIGURE 3.6 – Page d’inscription et définition des catégories

3.5.3 Tableau de bord

Après authentification, l'utilisateur accède à un tableau de bord présentant une synthèse de ses dépenses.

Cette interface permet notamment de consulter le montant total des dépenses, le nombre de factures enregistrées, la répartition des dépenses par catégorie ainsi que leur évolution mensuelle.

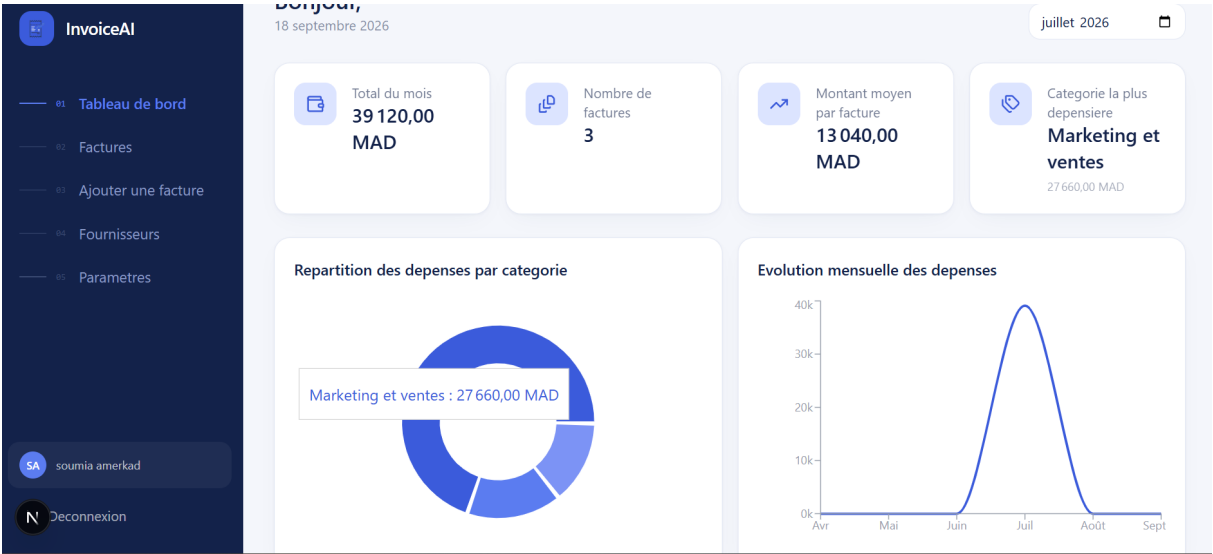


FIGURE 3.7 – Tableau de bord

3.5.4 Ajout d’une facture

L’interface d’ajout permet à l’utilisateur de téléverser une facture sous forme de fichier PDF ou image.

Après traitement par le modèle Gemini, les informations extraites sont présentées à l’utilisateur. Celui-ci peut les vérifier et les corriger avant de confirmer l’enregistrement de la facture.



The screenshot displays the InvoiceAI application interface. On the left is a dark blue sidebar with a menu containing: 'Tableau de bord', 'Factures', 'Ajouter une facture', 'Fournisseurs', and 'Parametres'. Below the menu are two buttons: 'SA soumia amerkad' and 'Deconnexion'. The central area shows a preview of an invoice from 'FIDUCOMPTABLE CONSEIL' dated '18/07/2026'. The invoice details include a table of services and a total amount of 6 120.00 MAD. On the right is a form for creating or editing an invoice, with fields for 'Fournisseur' (FIDUCOMPTABLE CONSEIL), 'N° Facture' (FC-2026-0789), 'Date' (18/07/2026), 'Montant HT' (5100 MAD), 'TVA (%)' (20), 'Montant TVA' (1020 MAD), 'Montant TTC' (6120 MAD), and 'Categorie' (Services professionnels). There are 'Annuler' and 'Confirmer' buttons at the bottom right.

InvoiceAI

- Tableau de bord
- Factures
- Ajouter une facture
- Fournisseurs
- Parametres

SA soumia amerkad

Deconnexion

Invoice Preview:

FIDUCOMPTABLE CONSEIL
Cabinet d'expertise comptable et conseil juridique
Boulevard Zerktouni, Casablanca - KSJ 905599778900012

FACTURE
N°: FC-2026-0789
Date: 18/07/2026

Facture n°:
Confexion Atlas SA
Prise en Relais, Km 12, Casablanca

Désignation	Quantité	Prix Unit. HT	Total HT
Tenue comptable mensuelle - Juillet	1 u	3500.00 MAD	3500.00 MAD
Conseil juridique - contrat fournisseur	2 h	800.00 MAD	1600.00 MAD
Montant HT:			5 100.00 MAD
TVA (20%):			1 020.00 MAD
Montant TTC:			6 120.00 MAD

Form Fields:

Fournisseur: FIDUCOMPTABLE CONSEIL

N° Facture: FC-2026-0789

Date: 18/07/2026

Montant HT: 5100 MAD

TVA (%): 20

Montant TVA: 1020 MAD

Montant TTC: 6120 MAD

Categorie: Services professionnels

Annuler Confirmer

FIGURE 3.8 – Ajout d’une facture avec extraction automatique

3.5.5 Liste des factures

Cette interface permet de consulter l'ensemble des factures enregistrées par l'utilisateur.

Des fonctionnalités de recherche et de filtrage permettent de retrouver rapidement une facture selon le fournisseur, la catégorie ou la période. L'utilisateur peut également modifier ou supprimer une facture.

SA

InvoiceAI

Factures

Toutes vos factures enregistrees

Q Rechercher un fournisseur...

Toutes les categories

Tous les mois

Tous les fournisseurs

Fournisseur	N° Facture	Date	Montant HT	TVA	Montant TTC	Categorie	Actions
PUBMEDIA AGENCY	PM-202...	20/07/2026	23 050,00 MAD	4 610,00 MAD	27 660,00 MAD	Marketing et v...	 
FIDUCOMPTABLE CONSEIL	FC-2026...	18/07/2026	5 100,00 MAD	1 020,00 MAD	6 120,00 MAD	Services profes...	 
BUREAUFIX SARL	BF-2026...	15/07/2026	4 450,00 MAD	890,00 MAD	5 340,00 MAD	Fournitures de ...	 

3 factures au total

SA

soumia amerkad

N

Deconnexion

FIGURE 3.9 – Liste des factures

3.5.6 Analyse par fournisseur

L’interface d’analyse par fournisseur permet de visualiser les dépenses associées aux différents fournisseurs. Elle fournit une synthèse des montants enregistrés et permet de générer un rapport Excel selon la période et le fournisseur sélectionnés.

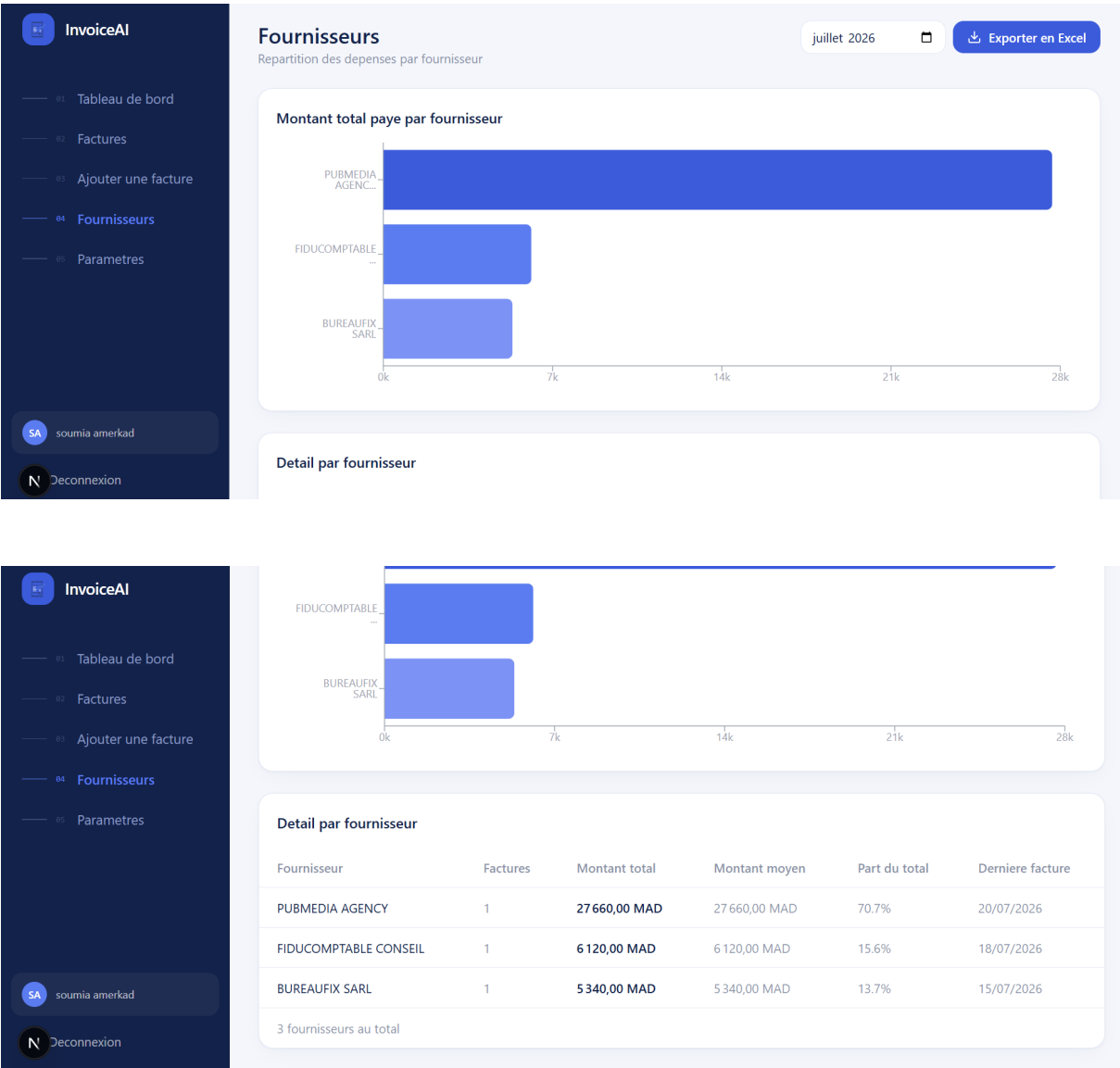


FIGURE 3.10 – Analyse des dépenses par fournisseur

3.6 Conclusion

Ce chapitre a présenté les outils, technologies et principales fonctionnalités mises en œuvre dans InvoiceAI, ainsi que les interfaces réalisées. Il a ainsi permis de présenter la

solution développée et son fonctionnement général.

Conclusion et Perspectives

Ce stage de six semaines m'a permis de concevoir et développer **InvoiceAI**, une application web d'automatisation de la gestion des factures fournisseurs par intelligence artificielle. Ce projet m'a permis d'approfondir mes compétences en développement full-stack (ASP.NET Core, Next.js), en intégration d'API d'intelligence artificielle, ainsi qu'en conception d'architectures sécurisées.

Plusieurs défis ont marqué ce projet : la conception d'un prompt d'extraction fiable, distinguant correctement les montants de ligne des totaux finaux d'une facture ; le choix motivé d'une plateforme d'intelligence artificielle parmi plusieurs alternatives ; et la conception d'un système de catégories entièrement personnalisables par l'utilisateur.

Plusieurs axes d'amélioration ont été identifiés pour une évolution future de l'application :

- Déploiement de l'application via Docker, et migration envisageable de la base de données vers Azure SQL Database.
- Passage à une offre payante de l'API Gemini pour un déploiement en production, garantissant la confidentialité des documents traités.

Ce projet constitue une base solide et évolutive pour l'automatisation de la gestion des factures fournisseurs, adaptable à différents secteurs d'activité grâce à son système de catégories personnalisables.

Webographie

1. Next.js Documentation, disponible sur : <https://nextjs.org/docs>
2. ASP.NET Core Documentation, disponible sur : <https://learn.microsoft.com/aspnet/core>
3. Entity Framework Core Documentation, disponible sur : <https://learn.microsoft.com/ef/core>
4. Google Gemini API Documentation, disponible sur : <https://ai.google.dev/gemini-api/docs>
5. JSON Web Tokens (JWT), disponible sur : <https://jwt.io/introduction>
6. ClosedXML Documentation, disponible sur : <https://github.com/ClosedXML/ClosedXML>

change this